

hw8

Katie Rhodes

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.1.0
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(forcats)
library(lubridate)
library(readr)
library(stringr)
library(rvest)
```

Attaching package: 'rvest'

The following object is masked from 'package:readr':

guess_encoding

```
library(dplyr)
library(nycflights13)
```

1) Use formula $S_n = (n)(n+1)(2n+1)/6$

```
compute_sumsq <- function(n) {
  n * (n+1) * (2*n+1)/6
}
compute_sumsq(n=15)
```

```
[1] 1240
```

```
compute_sumsq(n=27)
```

```
[1] 6930
```

2)

```
url <- "https://www.boxofficemojo.com/year/2024/"
movie_table <- read_html(url) |>
  html_elements("table") |>
  html_table() |>
  pluck(1) |>
  mutate(
    Gross = parse_number(Gross),
    Theaters = parse_number(Theaters),
    `Total Gross` = parse_number(`Total Gross`),
    `Release Date` = lubridate::mdy(str_glue("{`Release Date`} 2024"))
  ) |>
  select(-Genre, -Budget, -`Running Time`, -Estimated)
```

- a) Write a function called `scrape_bomojo()` that scrapes information about the domestic box office performance in a specified year. This function should do the following:

have one input: the URL of the webpage return a data frame rename variables scraped, using `snake_case` (Hint: consider the `janitor::clean_names()` function)

```
scrape_bomojo <- function(url){
  page <- read_html(url)
  movie_table <- page |>
    html_elements("table") |>
    html_table() |>
    pluck(1) |>
    janitor::clean_names() |>
    mutate(
      gross = parse_number(gross),
```

```

    total_gross = parse_number(total_gross),
    theaters = parse_number(theaters),
    month = word(release_date, 1),
    day = parse_number(release_date))
}

print(scrape_bomojo("https://www.boxofficemojo.com/year/2024/"))

```

```

# A tibble: 200 x 13
  rank release          genre budget running_time gross theaters total_gross
  <int> <chr>          <chr> <chr> <chr>      <dbl>    <dbl>    <dbl>
1     1 Inside Out 2      -    -    -      6.53e8    4440    652980194
2     2 Deadpool & Wolve~ -    -    -      6.37e8    4330    636745858
3     3 Wicked            -    -    -      4.33e8    3888    473231120
4     4 Moana 2           -    -    -      4.04e8    4200    460405297
5     5 Despicable Me 4    -    -    -      3.61e8    4449    361004205
6     6 Beetlejuice Beet~ -    -    -      2.94e8    4575    294100435
7     7 Dune: Part Two     -    -    -      2.82e8    4074    282144358
8     8 Twisters          -    -    -      2.68e8    4170    267762265
9     9 Godzilla x Kong:~ -    -    -      1.96e8    3948    196350016
10    10 Kung Fu Panda 4   -    -    -      1.94e8    4067    193590620
# i 190 more rows
# i 5 more variables: release_date <chr>, distributor <chr>, estimated <chr>,
#   month <chr>, day <dbl>

```

b)

```

scrape_bomojo2 <- function(year) {
  url <- str_c("https://www.boxofficemojo.com/", year, "/")
  page <- read_html(url)
  movie_table <- page |>
    html_elements("table") |>
    html_table() |>
    pluck(1) |>
    janitor::clean_names() |>
    mutate(
      gross = parse_number(gross),
      total_gross = parse_number(total_gross),
      theaters = parse_number(theaters),
      month = word(release_date, 1),
      day = parse_number(release_date))
}

```

```
}
```

3) Using the datasets from nycflights13, write a function that:

- a) Finds all flights that were cancelled (i.e. is.na(arr_time)) or delayed by more than an hour.

```
flights_severe <- function(flights) {  
  flights |>  
    filter(is.na(arr_time))  
}  
severe_flights <- flights |> flights_severe()
```

- b) Counts the number of cancelled flights and the number of flights delayed by more than an hour.

```
summarize_b <- function(flights) {  
  flights |>  
    summarize(cancelled = sum(is.na(arr_time)), delayed_1hr = sum(arr_delay > 60, na.rm = TRUE))  
}  
  
b <- flights |>  
  group_by(dest) |>  
  summarize_b()
```

- c) Finds all flights that were cancelled or delayed by more than a user supplied number of hours:

```
filter_severe <- function(flights, hours) {  
  min_hour <- hours * 60  
  flights |>  
    filter(is.na(arr_time) | arr_delay > min_hour)  
}  
c <- flights |> filter_severe(hours = 2)
```

- d) Summarizes the weather to compute the minimum, mean, and maximum, of a user supplied variable:

```

summarize_variable_d <- function(flights, var) {
  flights |>
    summarize(
      min_val = min(flights[[var]], na.rm = TRUE),
      mean_val = mean(flights[[var]], na.rm = TRUE),
      max_val = max(flights[[var]], na.rm = TRUE)
    )
}

```

- e) Converts the user supplied variable that uses clock time (e.g., dep_time, arr_time, etc.) into a decimal time (i.e. hours + (minutes / 60)).

```

standardize_time <- function(flights, time_var) {
  flights[[time_var]] <- (flights[[time_var]] %/% 100) + ((flights[[time_var]] %% 100) / 60)
}
e <- flights |> standardize_time("sched_dep_time")

```

4)

```

commute <- read.csv("http://aloy.rbind.io/data/CommuteAtlanta.csv")

```

The Time column contains the commute times of these 500 individuals.

Your task is to run a one-sample bootstrap and calculate a 95% percentile confidence interval for the average commute time in Atlanta, GA. To do this, write a for loop that does the following:

Use the `dplyr::slice_sample()` function to select a random sample of rows from the data frame, with replacement. Be sure that your data set for the random sample has the same number of rows as the original data set. In addition, don't hesitate to look at the help file for `dplyr::slice_sample()` to see how to specify the sample size and ensure that you are sampling with replacement. From this resampled data set, calculate the sample mean commute time and store the result. Repeat steps 1-2 1,000 times. Once you have your 1,000 sample means calculated, calculate the 0.025 and 0.975 quantiles of this vector using the `quantile()` function. This is a 95% percentile confidence interval.

```

n <- nrow(commute)
boot_means <- numeric(1000)
for (i in 1:1000) {
  sample <- commute |>
    slice_sample(n=n, replace = TRUE)
}

```

```
boot_means[i] <- mean(sample$Time)
}
quantile(boot_means, 0.025)
```

```
2.5%
27.4278
```

```
quantile(boot_means, 0.975)
```

```
97.5%
30.88115
```

- 5) A random walk proceeds as follows: A walker chooses a starting point on a line and then decides to take one step right or left at random, dependent on the toss of a fair coin (e.g., 50% chance of heads, 50% chance of tails). If the coin is HEADS, they move right, if it comes up TAILS then the move is to the left. The second step proceeds in the same manner, and so on for each subsequent step.

Your task is to use a for loop and if-else statements to implement a random walk and plot the first 100 steps the walker takes. To do this, here are a few hints:

Start the walker at ($x = 0$, $y = 0$), and use x to track the number of steps and y to track whether they are going to the left (-1) or right (+1).

You can use `sample(c("heads", "tails"), size = 1)` to flip a fair coin a single time.

The plot you construct should look similar to the plot below, though of course it will be different since these are random processes!

```
n_steps <- 100
x <- 0:n_steps
y <- numeric(n_steps + 1)
y[1] <- 0

for (i in 2:(n_steps + 1)) {
  coin <- sample(c("heads", "tails"), size = 1)
  if (coin == "heads") {
    step <- 1 }
  else {
    step <- -1 }
  y[i] <- y[i-1] + step
```

```
}  
walk_random <- data.frame(Step = x, Position = y)  
ggplot(walk_random, aes(x = Step, y = Position)) + geom_point() + geom_line()
```

